Dive into Qwen3.5 Key Techniques

From Linear Attention to Gated DeltaNet Hybrid Architecture

Presenter: Hung-Chun Hsu

CFDA, CLIP & LKT Labs Joint Laboratory Seminar
Research Center for Information Technology Innovation
Academia Sinica, Taipei, Taiwan

Apr 10, 2026
May 8, 2026
May 15, 2026

About Me

- **Name**: Hung-Chun Hsu
- **Title**: Research Assistant, CFDA Lab
- **Education**: M.S. in Data Science, National Taiwan University
- **Research Interests**: GNN, LLM, Multimodal-LLM, Information Retrieval
- **Interests**: Guitar, Physics, Pure Mathematics, exploring new things

r10946017@citi.sinica.edu.tw Google Scholar Website GitHub

Seminar Content Overview (I)

Week 1: Qwen3.5 — Gated DeltaNet Hybrid Architecture

1. Qwen3.5 Overview
   - Nobody Agrees on Attention Anymore
2. Linear Transformer
   - From Softmax to State Matrix; Blind Accumulation
3. DeltaNet: Delta Rule + Householder Erasure
   - "same old same old" example → DeltaNet derivation
4. Gated DeltaNet: DeltaNet + Mamba-2
5. Comparison of Three Methods
   - Equation ancestry + component comparison
6. Limitations + Hybrid Architecture
   - 3:1 hybrid surpasses Transformer++
7. Qwen3.5 Deployment & Comparisons

Seminar Content Overview (II)

Week 2: Qwen3.5 Native Multimodal & Gemma 4 PLE & IR Papers

1. Qwen3.5 native multimodal design
   - Early Fusion vs Late Fusion
2. Per-Layer Embeddings (PLE)
   - Gemma 4 edge model architecture
   - PLE effective params vs MoE active params
3. Recent IR papers & datasets
   - TREC-related work

Week 3 (Planning): DeepSeek Sparse Attention & Attention Sink

1. Attention Sink problem
   - StreamingLLM (ICLR 2024)
2. DeepSeek Native Sparse Attention
   - How it addresses Attention Sink

References and Acknowledgements I

Reference Papers:

- **Week 1**
  - ICLR 2025 — *Gated DeltaNet* — Yang, Kautz, Hatamizadeh
  - ICML 2021 — *DeltaNet* — Schlag, Irie, Schmidhuber
  - ICML 2024 — *Mamba-2* — Dao & Gu
  - 2026 — *Qwen3.5 Blog Post* — Alibaba Qwen Team (technical report not published yet)
- **Week 2**
  - 2026 — *Gemma 4 Model* — Google DeepMind
  - 2026 — *Qwen3.5 Native Multimodal* — Alibaba Qwen Team
  - arXiv 2026 — *Per-Layer Embeddings (PLE)* — Sadhukhan et al.
  - NeurIPS 2023 — *AltUp* — Baykal et al.
- **Week 3**
  - 2025 — *Native Sparse Attention* — DeepSeek AI
  - ICLR 2024 — *StreamingLLM* — Xiao et al.

References and Acknowledgements II

Acknowledgements:

- Special thanks to Bilibili — 五道口納什* (西安電子科技大學) for content inspiration
- Jia-Bin Huang (Professor, UMD) — awesome videos about latest AI methods on YouTube: @jbhuang0604
- Songlin Yang (Thinking Machines Lab, PhD MIT) — first author of Gated DeltaNet (ICLR 2025), creator of flash-linear-attention, and author of the DeltaNet Explained blog series
- Nano-banana
- Claude Code

* 五道口：北京海淀區地段，清華、北大所在；納什：數學家 John Nash，以「納什均衡」聞名。

Week 1 Outline (1/2) — Foundations

1. **Qwen3.5 Overview**
   - Qwen Model Family Timeline; Key Technical Innovations; Nobody Agrees on Attention Anymore
2. **Linear Transformer**
   - From Softmax to Decomposable Kernel → State Matrix → Recurrence
   - Coefficient matrix ($Y = V \cdot C^\top$); Linear vs Softmax comparison
   - Blind accumulation: "same old same old" (ABAB) example
3. **DeltaNet: Delta Rule + Householder Erasure**
   - From "same old same old" to DeltaNet's state update derivation
   - Error correction form → Householder matrix form
   - Linear Transformer vs DeltaNet comparison
4. **Gated DeltaNet: Combining DeltaNet + Mamba-2**
   - α_t global decay (Mamba-2) + β_t delta rule (DeltaNet)
   - Mamba-2 elements (Short Conv, SSD Duality)

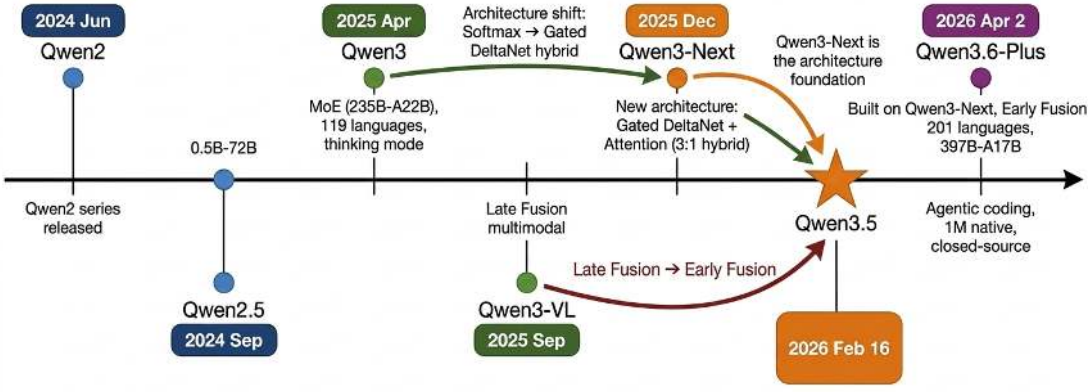
Week 1 Outline (2/2) — Comparison & Results

5. **Comparison of Three Methods**
   - Gated DeltaNet equation ancestry
   - State update equation evolution; component comparison
6. **Limitations of Linear Attention Layers**
   - Why not 100% Gated DeltaNet? In-context retrieval limitations
7. **Hybrid Architecture & Qwen3.5 Deployment**
   - 3:1 hybrid ratio surpassing Transformer++
   - Qwen3 vs Qwen3.5 architecture; Qwen3.5 vs Qwen3.6-Plus vs Claude Opus 4.5
8. **Supplementary Slides**
   - Mamba-2 technical details (SSD Duality, SSM unrolled time steps)
   - DeltaNet kernel conversion; State Matrix = Key-Value Memory

1. Qwen3.5 Overview

▶ 1. Qwen3.5 Overview
2. Linear Transformer
3. DeltaNet: Delta Rule + Householder Erasure
4. Gated DeltaNet: Combining DeltaNet + Mamba-2
5. Comparison of Three Methods
6. Limitations of Linear Attention Layers
7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

Qwen Model Family Timeline



Model Family Size Comparison

- Qwen2 (Blue): up to 72B dense
- Qwen2.5 (Blue): up to 72B dense
- Qwen3 (Green): up to 235B MoE (22B active)
- Qwen3-VL (Green): multimodal extension of Qwen3
- Qwen3-Next (Orange): architecture prototype (Gated DeltaNet)
- Qwen3.5 (Orange): up to 397B MoE (17B active) + 0.8B-27B dense
- Qwen3.6-Plus (Purple): closed-source API only

Key Technical Innovations in Qwen3.5

#	Technique	Description	Original?
1	**Gated DeltaNet + Gated Attention**	3:1 hybrid ratio (75% linear + 25% softmax)	**Nearly first** — ICLR 2025 (NVIDIA), Kimi adopted concurrently
2	**Native Multimodal (Early Fusion)**	Text/image/video jointly processed from pre-training (Week 2)	No — Llama 4 (2025/04) adopted ∼10 months earlier
3	**Sparse MoE**	397B total, 17B active	No — Mixtral (2023/12), DeepSeek-V2 (2024/05)
4	**Inference Performance Boost**	32K: 8.6x↑, 256K: 19x↑	— (architecture combination)
5	**Ultra-Long Context**	Native 262K, extensible to 1M	No — Llama 4 Scout (10M), Gemma 4 (256K)
6	**201 Languages / 250K Vocab**	Broadest multilingual in open-source	**Leading** among open-source
7	**Multi-Token Prediction**	Multi-token per step, lower cost	No — DeepSeek-V3 (2024/12)
8	**RL Training Innovation**	Expanding RL env difficulty & generalization	Hard to compare — details undisclosed

Nobody Agrees on Attention Anymore

The divergence has shifted from "MoE or Dense?" to the **attention mechanism itself**:

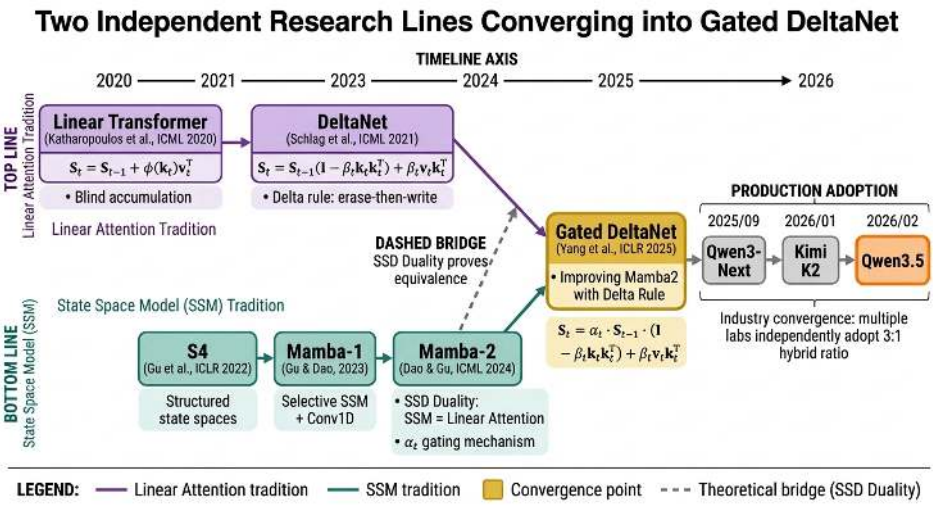
Model	Released	Attention Mechanism	Approach
DeepSeek-V3	2024/12	MLA + Native Sparse Attention (NSA)	Low-rank + sparse
Kimi K2.5	2025/06	Multi-head Latent Attention (MLA)	Low-rank compression
MiniMax-M2.5	2025/08	Standard Multi-Head Attention (MHA)	Full attention
GLM-5	2025/12	MLA + DeepSeek Sparse Attention (DSA)	Low-rank + sparse
Qwen3.5	2026/02	Gated DeltaNet + Full Attention (3:1)	Linear attention hybrid

- DeepSeek's MLA/DSA widely adopted (Kimi, GLM)
- Qwen3.5 opens a new direction: **linear attention** replacing most softmax layers

References:

https://huggingface.co/blog/mlabonne/qwen35
https://qwen.ai/blog?id=qwen3.5

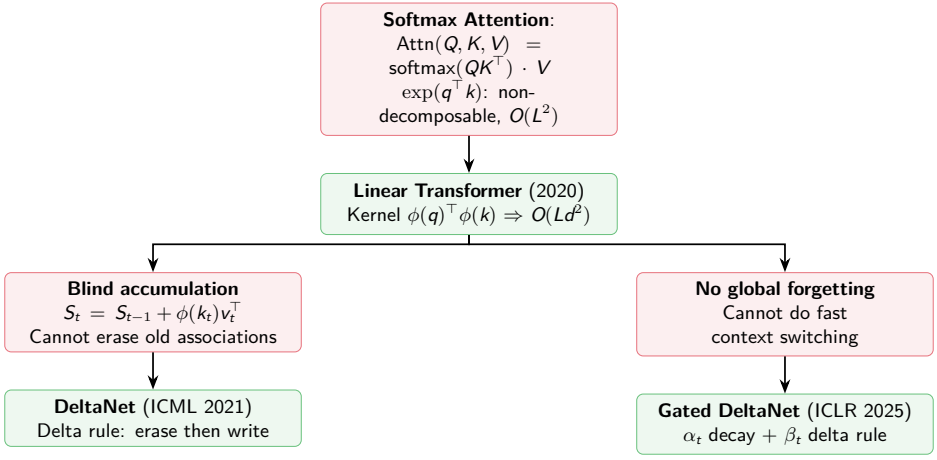
Week 1 Focus — Qwen3.5: Gated DeltaNet



Note: SSD = Structured State-Space Duality (Mamba-2's central theoretical contribution, ICML 2024).

Motivation: From Softmax Attention to Gated DeltaNet

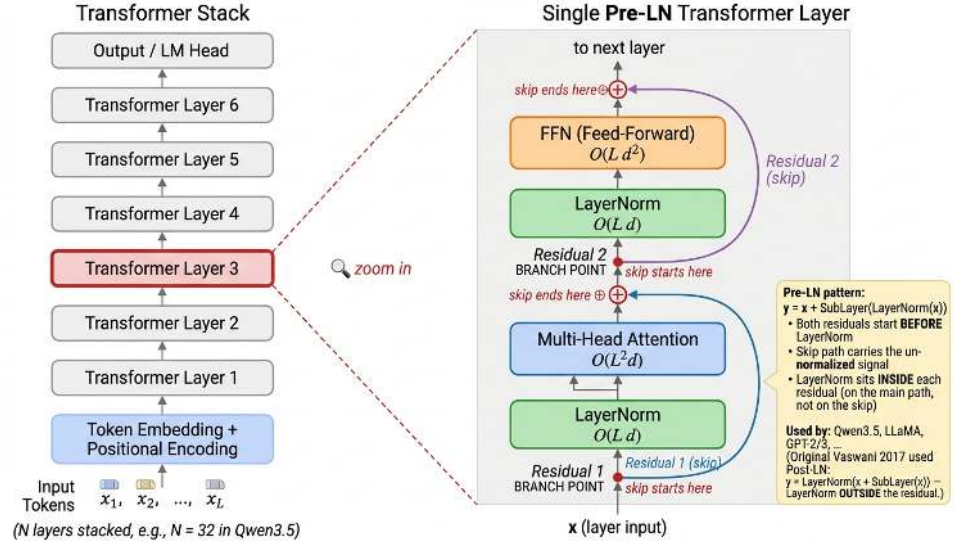
The problem: Softmax attention is $O(L^2 d)$ — quadratic in sequence length.



2. Linear Transformer

1. Qwen3.5 Overview
▶ **2. Linear Transformer**
3. DeltaNet: Delta Rule + Householder Erasure
4. Gated DeltaNet: Combining DeltaNet + Mamba-2
5. Comparison of Three Methods
6. Limitations of Linear Attention Layers
7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

Background — Transformer Layer Components & Complexity



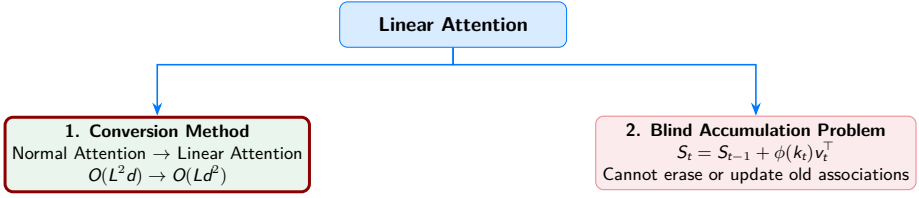
Linear Transformer — Background

Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention

Authors: Katharopoulos, Vyas, Pappas, Fleuret **Venue:** ICML 2020
Key idea: Replace softmax with a kernel feature map ϕ, enabling associativity $\Rightarrow$ rewrite attention as an RNN with a fixed-size state matrix.

The problem: Standard Transformer attention has $O(L^2 d)$ complexity — **quadratic** in sequence length L.

Two key topics in this section:



Linear Attention — From Softmax to a Decomposable Kernel

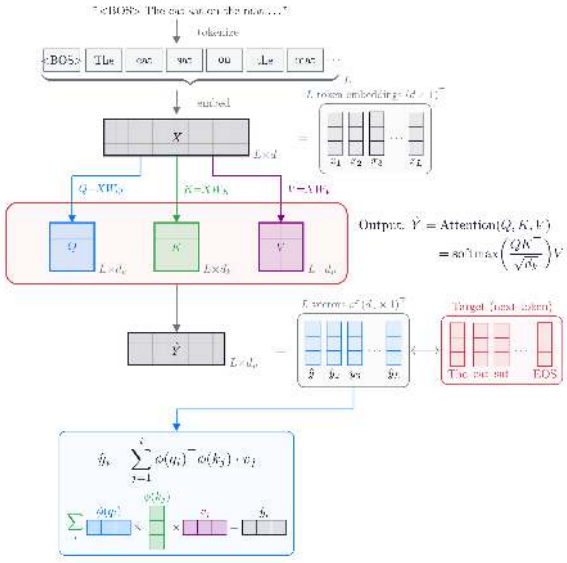
Recap — standard attention layer. Given $Q, K, V \in \mathbb{R}^{L \times d}$ (one row per token):

$$\mathrm{Attn}(Q,K,V) \;=\; \underbrace{\mathrm{softmax}\Big(\overbrace{\frac{QK^\top}{\sqrt{d_k}}}^{O(L^2 d)}\Big)\,V}_{O(L^2 d)\ \text{(multiply by } V)}.$$

Reading the i-th row with a causal mask ($j \le i$) gives a weighted sum of past values:

$$y_i \;=\; \sum_{j=1}^{i} \alpha_{ij}\, v_j, \qquad \alpha_{ij} \;=\; \frac{\overbrace{\exp(q_i^\top k_j/\sqrt{d_k})}^{O(d)}}{\sum_{l=1}^{i} \exp(q_i^\top k_l/\sqrt{d_k})}.$$

One y_i: L inner products $\times\, O(d) = O(L \cdot d)$. **All** $y_1, \ldots, y_L$: $L \times O(L \cdot d) = O(L^2 d)$.



Linear Attention — Where the $O(L^2 d)$ Cost Comes From

Dropping $1/\sqrt{d_k}$ for brevity:

$$y_i = \frac{\sum_{j=1}^{L} \overbrace{\exp(q_i^\top k_j)}^{O(d)} \, v_j}{\sum_{j=1}^{L} \exp(q_i^\top k_j)}$$

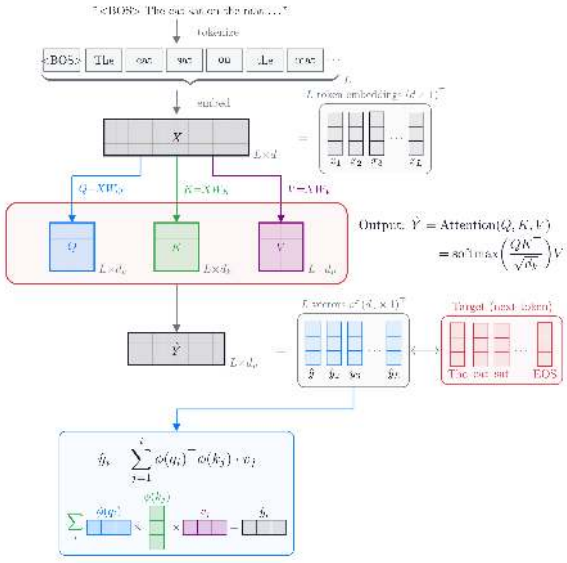
Causal form: $y_i = \frac{\sum_{j=1}^{i} \exp(q_i^\top k_j)\, v_j}{\sum_{j=1}^{i} \exp(q_i^\top k_j)}$ (attend only to $j \le i$)

One y_i: L inner products $\times$ $O(d)$ $=$ $O(L \cdot d)$

All $y_1, \ldots, y_L$:

$$L \times O(L \cdot d) = O(L^2 d)$$

Bottleneck: $\exp(q_i^\top k_j)$ is **non-decomposable** — q_i cannot be factored out of the sum over j.



Linear Attention — From Softmax to a Decomposable Kernel (cont.)

Key move: replace $\exp(q^\top k)$ with a **decomposable kernel** $\phi(q)^\top \phi(k)$ for some finite-dim $\phi : \mathbb{R}^d \to \mathbb{R}^d$:

$$y_i = \frac{\sum_{j=1}^{i} \phi(q_i)^\top \phi(k_j)\, v_j}{\sum_{j=1}^{i} \phi(q_i)^\top \phi(k_j)} \propto \sum_{j=1}^{i} \phi(q_i)^\top \overbrace{\phi(k_j)}^{\mathbb{R}^d} \cdot \overbrace{v_j}^{\mathbb{R}^d}$$

$$= \overbrace{\phi(q_i)^\top}^{\text{factor out}} \underbrace{\sum_{j=1}^{i} \phi(k_j)\, v_j^\top}_{\substack{\text{state matrix } S_i \in \mathbb{R}^{d\times d} \\ \text{build once: } O(L\cdot d^2)}}$$

One $y_i = \phi(q_i)^\top S_i$: $O(d^2)$ per query. **All** $y_1, \ldots, y_L$: $L \times O(d^2) = O(L \cdot d^2)$ — **linear in** L.

$\phi(q_i)$ factors out $\Rightarrow$ S_i is **accumulated once, reused for all queries**; total drops from $O(L^2 d)$ to $O(L d^2)$.

But wait — can we just swap $\exp$ for any function we want? Is $\phi(q)^\top \phi(k)$ a "valid" attention similarity? Next slide.

Why is this Replacement Legal? — Mercer's Theorem

Tsai et al. (2019), *Transformer Dissection*, EMNLP — attention as a kernel smoother (核平滑器)

$$\text{Attn}_i = \frac{\sum_j K(q_i, k_j)\, v_j}{\sum_j K(q_i, k_j)}, \qquad K(q, k) = \exp(q^\top k/\sqrt{d})$$

The kernel K decides "who matches"; $\exp$ is just **one** valid PSD kernel — Mercer's theorem says we can swap it.

Mercer's theorem. Any continuous, symmetric, positive semi-definite (PSD) kernel $K(q, k)$ equals an inner product in some Hilbert space:

$$K(q, k) = \langle \Phi(q), \Phi(k) \rangle_{\mathcal{H}}.$$

What Mercer gives us	**Consequence for linear attention**
$\exp(q^\top k)$ is PSD	Taylor expansion + Schur product theorem ⇒ softmax was already a Mercer kernel
Any $\phi(q)^\top \phi(k)$ is PSD	**Free pass to design any finite-dim ϕ** — no need to re-verify legality
Replacement on previous slide is legal	ELU+1, DPFP, SiLU all auto-legitimate

Bottom line: $\exp$ is **one** kernel among infinitely many. We can freely pick a finite-dim ϕ to enable the linearization on the previous slide, with full theoretical backing from a 1909 result.

Complexity & Why Softmax Can't Linearize Itself

Why can't softmax do the same trick? $\exp(q^\top k)$ corresponds to an **infinite-dimensional** feature map (Taylor expansion):

$$\Phi_{\exp}(x) = \left[1,\ \tfrac{x_1}{\sqrt{1!}},\ \ldots,\ \tfrac{x_d}{\sqrt{1!}},\ \tfrac{x_i x_j}{\sqrt{2!}},\ \ldots\right] \in \mathcal{H}_\infty \ \Rightarrow\ S = \sum_j \Phi_{\exp}(k_j)\, v_j^\top \in \mathbb{R}^{\infty \times d_v}$$

Complexity:

	Computation Order	Intermediate Matrix	Cost
Standard	$(QK^\top) \cdot V$	$\underbrace{QK^\top}_{L \times L}$	$O(L^2 d)$
Linear	$Q \cdot (K^\top V)$	$\underbrace{K^\top V}_{d' \times d_v}$	$O(L d' d_v)$

When $L \gg d$, the speedup is dramatic. Linear attention trades the $L \times L$ matrix for a $d' \times d_v$ one.

Choices of ϕ in the literature:

Model	Kernel ϕ
Linear Transformer (2020)	ELU+1
DeltaNet (2021)	DPFP (ReLU variant)
Gated DeltaNet (2025)	SiLU
Qwen3-Next / Qwen3.5	SiLU

Linear Attention — State Matrix & Recurrence

Define the **state matrix** from the precomputable sum:

$$\underbrace{y_i = \phi(q_i)^\top \sum_{j=1}^{i} \phi(k_j) v_j^\top}_{\text{from previous slide}} \quad \Longrightarrow \quad \boxed{S_i \triangleq \sum_{j=1}^{i} \phi(k_j) v_j^\top} \quad \Longrightarrow \quad \boxed{y_i = S_i \cdot \phi(q_i)}$$

Rewrite as recurrence (split off the last term):

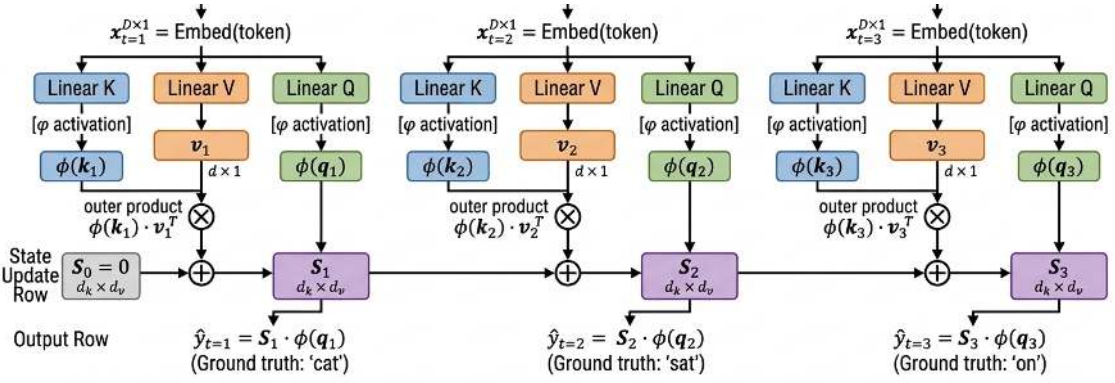
$$S_i = \underbrace{\sum_{j=1}^{i-1} \phi(k_j) v_j^\top}_{=S_{i-1}} + \phi(k_i) v_i^\top \quad \Longrightarrow \quad \boxed{S_i = S_{i-1} + \underbrace{\phi(k_i) \cdot v_i^\top}_{\text{outer product}}}$$

Each step: add one outer product to S. No need to look back at history.
Fixed-size $S \in \mathbb{R}^{d_k \times d_v}$ replaces the growing KV cache $\in \mathbb{R}^{L \times d}$.

Linear Attention — State Matrix Recurrence

Example: "The cat sat on the mat ..."

$\underbrace{\text{The}}_{t=1}$ $\underbrace{\text{cat}}_{t=2}$ $\underbrace{\text{sat}}_{t=3}$ on the mat ..."



Linear Attention Recurrence:

$$S_t = S_{t-1} + \phi(k_t) \cdot v_t^T \qquad \hat{y}_t = S_t \cdot \phi(q_t)$$

Symbol	Meaning		
$\phi(q_t)$	Transformed query vector	v_t	Value vector
$\phi(k_t)$	Transformed key vector	$\phi()$	Kernel function (e.g. ReLU, ELU+1, SiLU)
v_t	Value vector	S_t	State matrix (memory), $d_k \times d_v$
S_t	Scope matrix (memory), $d_k \times d_v$	$\hat{y}_t$	Output (prediction)

Linear Attention — Comparison with Softmax Attention

	Softmax Attention	**Linear Attention**
Similarity fn	$\exp(q^\top k)$ (non-decomposable)	$\phi(q)^\top \phi(k)$ (decomposable)
Factor out q_i?	No $\rightarrow$ must compute pairwise	**Yes** $\rightarrow$ state matrix emerges
Memory mech.	KV cache (stores k, v per position)	State matrix S (fixed-size matrix)
Query mech.	Attends to all historical positions	Directly queries S with $\phi(q)$
Memory size	$O(L \cdot d)$ (grows with length)	$O(d_k \times d_v)$ (fixed)
Complexity	$O(L^2 d)$	$O(Ld^2)$

State Evolution — Example with a Length-4 Sequence

Notation: $\mathbf{q}_t$, $\mathbf{k}_t$ denote $\phi(\mathbf{q}_t)$, $\phi(\mathbf{k}_t)$ for brevity. **Recurrence:** $\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(\mathbf{k}_t)\,\mathbf{v}_t^\top = \mathbf{S}_{t-1} + \mathbf{v}_t\,\mathbf{k}_t^\top$, $\mathbf{S}_0 = \mathbf{0}$.

	$t=1$	$t=2$	$t=3$	$t=4$
input:	x_1	x_2	x_3	x_4
	↓	↓	↓	↓
target:	x_2	x_3	x_4	$\langle eos \rangle$

State evolution ($\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\,\mathbf{k}_t^\top$):

$$t{=}0: \quad \mathbf{S}_0 = \mathbf{0}$$

$$t{=}1: \quad \mathbf{S}_1 = \mathbf{S}_0 + \mathbf{v}_1\,\mathbf{k}_1^\top = \mathbf{v}_1\,\mathbf{k}_1^\top$$

$$t{=}2: \quad \mathbf{S}_2 = \mathbf{S}_1 + \mathbf{v}_2\,\mathbf{k}_2^\top = \mathbf{v}_1\,\mathbf{k}_1^\top + \mathbf{v}_2\,\mathbf{k}_2^\top$$

$$t{=}3: \quad \mathbf{S}_3 = \mathbf{S}_2 + \mathbf{v}_3\,\mathbf{k}_3^\top = \mathbf{v}_1\,\mathbf{k}_1^\top + \mathbf{v}_2\,\mathbf{k}_2^\top + \mathbf{v}_3\,\mathbf{k}_3^\top$$

$$t{=}4: \quad \mathbf{S}_4 = \mathbf{S}_3 + \mathbf{v}_4\,\mathbf{k}_4^\top = \mathbf{v}_1\,\mathbf{k}_1^\top + \mathbf{v}_2\,\mathbf{k}_2^\top + \mathbf{v}_3\,\mathbf{k}_3^\top + \mathbf{v}_4\,\mathbf{k}_4^\top$$

Recurrence:

$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\,\mathbf{k}_t^\top$

$\mathbf{S}_0 = \mathbf{0}, \quad \mathbf{S} \in \mathbb{R}^{d \times d}$ fixed-size

Linear projections (frozen):

$\mathbf{v}_t = \mathbf{W}_V\,\mathbf{x}_t, \quad \mathbf{k}_t = \mathbf{W}_K\,\mathbf{x}_t, \quad \mathbf{q}_t = \mathbf{W}_Q\,\mathbf{x}_t$

$\mathbf{W}_V, \mathbf{W}_K, \mathbf{W}_Q$ are **fixed** during inference.

Decoding — Substituting State & Applying Associativity

Decoding: $\mathbf{y}_t = \mathbf{S}_t\,\mathbf{q}_t$. Substituting the state and applying associativity $(\mathbf{v}_j\,\mathbf{k}_j^\top)\,\mathbf{q}_t = (\mathbf{k}_j^\top\,\mathbf{q}_t)\,\mathbf{v}_j$:

$$
\begin{aligned}
\mathbf{y}_1 &= \mathbf{S}_1\,\mathbf{q}_1 = (\mathbf{v}_1\,\mathbf{k}_1^\top)\,\mathbf{q}_1 &&= (\mathbf{k}_1^\top\,\mathbf{q}_1)\,\mathbf{v}_1 \\
\mathbf{y}_2 &= \mathbf{S}_2\,\mathbf{q}_2 = (\mathbf{v}_1\,\mathbf{k}_1^\top + \mathbf{v}_2\,\mathbf{k}_2^\top)\,\mathbf{q}_2 &&= (\mathbf{k}_1^\top\,\mathbf{q}_2)\,\mathbf{v}_1 + (\mathbf{k}_2^\top\,\mathbf{q}_2)\,\mathbf{v}_2 \\
\mathbf{y}_3 &= \mathbf{S}_3\,\mathbf{q}_3 = (\mathbf{v}_1\,\mathbf{k}_1^\top + \mathbf{v}_2\,\mathbf{k}_2^\top + \mathbf{v}_3\,\mathbf{k}_3^\top)\,\mathbf{q}_3 &&= (\mathbf{k}_1^\top\,\mathbf{q}_3)\,\mathbf{v}_1 + (\mathbf{k}_2^\top\,\mathbf{q}_3)\,\mathbf{v}_2 + (\mathbf{k}_3^\top\,\mathbf{q}_3)\,\mathbf{v}_3 \\
\mathbf{y}_4 &= \mathbf{S}_4\,\mathbf{q}_4 = (\mathbf{v}_1\,\mathbf{k}_1^\top + \mathbf{v}_2\,\mathbf{k}_2^\top + \mathbf{v}_3\,\mathbf{k}_3^\top + \mathbf{v}_4\,\mathbf{k}_4^\top)\,\mathbf{q}_4 &&= (\mathbf{k}_1^\top\,\mathbf{q}_4)\,\mathbf{v}_1 + (\mathbf{k}_2^\top\,\mathbf{q}_4)\,\mathbf{v}_2 + (\mathbf{k}_3^\top\,\mathbf{q}_4)\,\mathbf{v}_3 + (\mathbf{k}_4^\top\,\mathbf{q}_4)\,\mathbf{v}_4
\end{aligned}
$$

Each output $\mathbf{y}_t$ is a **linear combination** of past values $\mathbf{v}_1, \ldots, \mathbf{v}_t$, weighted by scalar key-query similarities $(\mathbf{k}_j^\top\,\mathbf{q}_t)$ — the linear attention analogue of softmax attention's weighted sum, but without normalization.

Coefficient Matrix — $n=4$ Instance

Stacking the equations from the previous slide into matrix form $Y = V \cdot C^\top$:

$$\underbrace{[\mathbf{y}_1,\ \mathbf{y}_2,\ \mathbf{y}_3,\ \mathbf{y}_4]}_{Y\ (d\times 4)} = \underbrace{[\mathbf{v}_1,\ \mathbf{v}_2,\ \mathbf{v}_3,\ \mathbf{v}_4]}_{V\ (d\times 4)} \cdot \underbrace{\begin{bmatrix} (\mathbf{k}_1^\top \mathbf{q}_1) & (\mathbf{k}_1^\top \mathbf{q}_2) & (\mathbf{k}_1^\top \mathbf{q}_3) & (\mathbf{k}_1^\top \mathbf{q}_4) \\ 0 & (\mathbf{k}_2^\top \mathbf{q}_2) & (\mathbf{k}_2^\top \mathbf{q}_3) & (\mathbf{k}_2^\top \mathbf{q}_4) \\ 0 & 0 & (\mathbf{k}_3^\top \mathbf{q}_3) & (\mathbf{k}_3^\top \mathbf{q}_4) \\ 0 & 0 & 0 & (\mathbf{k}_4^\top \mathbf{q}_4) \end{bmatrix}}_{C^\top\ (4\times 4)}$$

$C^\top$ is **upper triangular**: column t contains only contributions from $j \le t$ (causal masking). Each entry $(\mathbf{k}_j^\top \mathbf{q}_t)$ is a scalar key-query similarity. Next: generalize to $n \times n$.

Coefficient Matrix — General Form

Stacking the n outputs as columns and reading off the scalar coefficients:

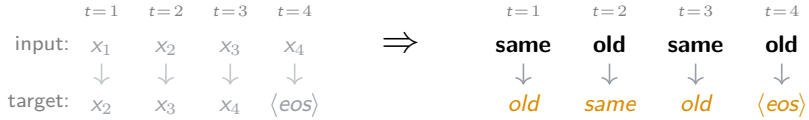
$$\underbrace{[\mathbf{y}_1,\, \mathbf{y}_2,\, \ldots,\, \mathbf{y}_n]}_{Y\ (d\times n)} = \underbrace{[\mathbf{v}_1,\, \mathbf{v}_2,\, \ldots,\, \mathbf{v}_n]}_{V\ (d\times n)} \cdot \underbrace{\begin{bmatrix} (\mathbf{k}_1^\top \mathbf{q}_1) & (\mathbf{k}_1^\top \mathbf{q}_2) & \cdots & (\mathbf{k}_1^\top \mathbf{q}_n) \\ 0 & (\mathbf{k}_2^\top \mathbf{q}_2) & \cdots & (\mathbf{k}_2^\top \mathbf{q}_n) \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & (\mathbf{k}_n^\top \mathbf{q}_n) \end{bmatrix}}_{C^\top\ (n\times n)}$$

- $C^\top$ is $n \times n$ **upper triangular** — the transpose of the lower-triangular causal coefficient matrix C.
- **Upper-triangular structure** = causal masking: column t of $C^\top$ contains only contributions from $j \le t$.
- Equivalently, row t of C gives the coefficients for $\mathbf{y}_t = \sum_{j=1}^{t} (\mathbf{k}_j^\top \mathbf{q}_t)\, \mathbf{v}_j$.

Compact form: $Y = V \cdot C^\top$. This matrix equation is the basis for both the **parallel computation** (training) and the **structural analysis** of self-duplication (next slides).

The Fundamental Problem — Substituting "same old same old"

Now substitute a concrete sequence. Consider the ABAB pattern **"same old same old"**:



$x_1 = x_3 =$ "same", $x_2 = x_4 =$ "old". Same input ⇒ same projections:

	$t=1$: same	$t=2$: old	$t=3$: same	$t=4$: old
Key	$\mathbf{k}_1$	$\mathbf{k}_2$	$\mathbf{k}_3 = \mathbf{k}_1$	$\mathbf{k}_4 = \mathbf{k}_2$
Value	$\mathbf{v}_1$	$\mathbf{v}_2$	$\mathbf{v}_3 = \mathbf{v}_1$	$\mathbf{v}_4 = \mathbf{v}_2$
Query	$\mathbf{q}_1$	$\mathbf{q}_2$	$\mathbf{q}_3 = \mathbf{q}_1$	$\mathbf{q}_4 = \mathbf{q}_2$

The system has **4 time steps** but only **2 independent inputs**. The state update $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ does not know this — it blindly adds all 4 outer products. What happens?

Coefficient Matrix with Target Labels

Substituting the ABAB constraints into the coefficient matrix, with target labels:

$$\underbrace{[\underbrace{\mathbf{y}_1}_{\text{“old”}}, \underbrace{\mathbf{y}_2}_{\text{“same”}}, \underbrace{\mathbf{y}_3}_{\text{“old”}}, \underbrace{\mathbf{y}_4}_{\text{“⟨EOS⟩”}}]}_{Y\ (d\times 4)} = \underbrace{[\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \mathbf{v}_4]}_{V\ (d\times 4)} \cdot \underbrace{\begin{bmatrix} (\mathbf{k}_1^\top \mathbf{q}_1) & (\mathbf{k}_1^\top \mathbf{q}_2) & (\mathbf{k}_1^\top \mathbf{q}_3) & (\mathbf{k}_1^\top \mathbf{q}_4) \\ 0 & (\mathbf{k}_2^\top \mathbf{q}_2) & (\mathbf{k}_2^\top \mathbf{q}_3) & (\mathbf{k}_2^\top \mathbf{q}_4) \\ 0 & 0 & (\mathbf{k}_3^\top \mathbf{q}_3) & (\mathbf{k}_3^\top \mathbf{q}_4) \\ 0 & 0 & 0 & (\mathbf{k}_4^\top \mathbf{q}_4) \end{bmatrix}}_{C^\top\ (4\times 4)}$$

Target labels on the left show what each $\mathbf{y}_t$ should map to. Note that $\mathbf{y}_1$ and $\mathbf{y}_3$ should both map to “old” — next we substitute the ABAB constraints to see if this expectation can be met.

Blind Accumulation — $\mathbf{y}_1 \neq \mathbf{y}_3$ but Both Should Be "old"

Substituting ABAB constraints ($\mathbf{k}_3 = \mathbf{k}_1$, $\mathbf{q}_3 = \mathbf{q}_1$, $\mathbf{v}_3 = \mathbf{v}_1$, $\mathbf{k}_4 = \mathbf{k}_2$, $\mathbf{q}_4 = \mathbf{q}_2$, $\mathbf{v}_4 = \mathbf{v}_2$):

$$\underbrace{[\underbrace{\mathbf{y}_1}_{\text{"old"}}, \underbrace{\mathbf{y}_2}_{\text{"same"}}, \underbrace{\mathbf{y}_3}_{\text{"old"}}, \underbrace{\mathbf{y}_4}_{\text{"⟨EOS⟩"}}]}_{Y\ (d\times 4)} = \underbrace{[\mathbf{v}_1, \mathbf{v}_2, \overbrace{\mathbf{v}_3}^{=\mathbf{v}_1}, \overbrace{\mathbf{v}_4}^{=\mathbf{v}_2}]}_{V\ (d\times 4)} \cdot \underbrace{\begin{bmatrix} (\mathbf{k}_1^\top \mathbf{q}_1) & (\mathbf{k}_1^\top \mathbf{q}_2) & (\mathbf{k}_1^\top \mathbf{q}_1) & (\mathbf{k}_1^\top \mathbf{q}_2) \\ 0 & (\mathbf{k}_2^\top \mathbf{q}_2) & (\mathbf{k}_2^\top \mathbf{q}_1) & (\mathbf{k}_2^\top \mathbf{q}_2) \\ 0 & 0 & (\mathbf{k}_1^\top \mathbf{q}_1) & (\mathbf{k}_1^\top \mathbf{q}_2) \\ 0 & 0 & 0 & (\mathbf{k}_2^\top \mathbf{q}_2) \end{bmatrix}}_{C^\top\ (4\times 4)}$$

Both $\mathbf{y}_1$ and $\mathbf{y}_3$ should map to "old", but reading rows 1 and 3:

$$\mathbf{y}_1 = (\mathbf{k}_1^\top \mathbf{q}_1)\, \mathbf{v}_1 \qquad \neq \qquad \mathbf{y}_3 = \boxed{2}\, (\mathbf{k}_1^\top \mathbf{q}_1)\, \mathbf{v}_1 + (\mathbf{k}_2^\top \mathbf{q}_1)\, \mathbf{v}_2$$

Remember: $\mathbf{y}_3 = \mathbf{S}_3\, \mathbf{q}_3$, where $\mathbf{S}_3 = \underbrace{\mathbf{S}_2}_{\mathbf{v}_1\mathbf{k}_1^\top + \mathbf{v}_2\mathbf{k}_2^\top} + \mathbf{v}_1\, \mathbf{k}_1^\top$ ➤ ($\mathbf{v}_1\, \mathbf{k}_1^\top$ added to $\mathbf{S}$ twice).

Self-duplication:

- $\mathbf{v}_1$'s coefficient doubles from 1 to 2 because the update $\mathbf{S} + \mathbf{v}\,\mathbf{k}^\top$ blindly added the same $\mathbf{v}_1\, \mathbf{k}_1^\top$ a second time.

Blind Accumulation — Why $\mathbf{y}_1 \neq \mathbf{y}_3$

Token "same" appears at $t=1$ and $t=3$. Both times the model should predict "old" — but linear attention gives **different answers**. Why?

Cause 1 — the state at $t=3$ contains a **duplicate** outer product:

$$\mathbf{S}_3 = \underbrace{\mathbf{S}_2}_{\text{previous state}} + \underbrace{\mathbf{v}_3\,\mathbf{k}_3^\top}_{=\mathbf{v}_1\,\mathbf{k}_1^\top} = \underbrace{\mathbf{v}_1\,\mathbf{k}_1^\top}_{t=1} + \mathbf{v}_2\,\mathbf{k}_2^\top + \underbrace{\mathbf{v}_1\,\mathbf{k}_1^\top}_{t=3:\ \text{duplicate}}$$

Consequence — $\mathbf{v}_1$'s coefficient doubles:

$$\mathbf{y}_1 = 1 \cdot (\mathbf{k}_1^\top \mathbf{q}_1)\,\mathbf{v}_1 \quad \neq \quad \mathbf{y}_3 = \underbrace{\boxed{2} \cdot (\mathbf{k}_1^\top \mathbf{q}_1)\,\mathbf{v}_1}_{\text{duplicate write} \to \text{coefficient doubled}} + (\mathbf{k}_2^\top \mathbf{q}_1)\,\mathbf{v}_2$$

Linear Transformer's update rule $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\,\mathbf{k}_t^\top$ is append-only:

- Cannot **detect** whether $\mathbf{k}_t$ already exists in state
- Cannot **erase** the old $\mathbf{v}_1\,\mathbf{k}_1^\top$ before writing the duplicate

Blind Accumulation — The Cross-Attention Term

Besides the duplicated $\mathbf{v}_1$ coefficient, $\mathbf{y}_3$ also contains a **cross-attention term** $(\mathbf{k}_2^\top \mathbf{q}_1)\,\mathbf{v}_2$.

Cause 2 — query $\mathbf{q}_1$ at $t{=}3$ attends to **all** past keys in $\mathbf{S}_3$, including $\mathbf{k}_2$:

$$\mathbf{S}_3 = \underbrace{\mathbf{v}_1\,\mathbf{k}_1^\top}_{t=1} + \underbrace{\mathbf{v}_2\,\mathbf{k}_2^\top}_{t=2} + \underbrace{\mathbf{v}_1\,\mathbf{k}_1^\top}_{t=3}$$

Consequence — $\mathbf{y}_3$ picks up $\mathbf{v}_2$ via cross-attention:

$$\mathbf{y}_1 = (\mathbf{k}_1^\top \mathbf{q}_1)\,\mathbf{v}_1 \quad \neq \quad \mathbf{y}_3 = \boxed{2} \cdot (\mathbf{k}_1^\top \mathbf{q}_1)\,\mathbf{v}_1 + \underbrace{(\mathbf{k}_2^\top \mathbf{q}_1)\,\mathbf{v}_2}_{\text{cross-attention term}}$$

Summary: Two conditions for $\mathbf{y}_1 = \mathbf{y}_3$:

- Avoid duplicated coefficient on $\mathbf{v}_1$: $\boxed{2} \to 1 \quad \longrightarrow$ **DeltaNet** (delta rule fixes this)
- Require $\mathbf{k}_2^\top \mathbf{q}_1 = 0 \quad \longrightarrow$ **Gated DeltaNet** introduces forgetting mechanism (α_t decay), but does not fully resolve this

3. DeltaNet: Delta Rule + Householder Erasure

1. Qwen3.5 Overview
2. Linear Transformer
▶ **3. DeltaNet: Delta Rule + Householder Erasure**
4. Gated DeltaNet: Combining DeltaNet + Mamba-2
5. Comparison of Three Methods
6. Limitations of Linear Attention Layers
7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

DeltaNet 的動機 — 修正重複 Key 的寫入行為

- DeltaNet 修正了 Linear Transformer 中「重複 key 寫入 state matrix」的行為。

「**same old same old**」問題回顧：在 $t = 3$ 時「same」再次寫入（$\mathbf{v}_3\,\mathbf{k}_3^\top = \mathbf{v}_1\,\mathbf{k}_1^\top$），但 state update 不知道 $\mathbf{k}_1$ 已存在：

$$\mathbf{S}_3^{\text{Linear Transformer}} = \mathbf{S}_2 + \mathbf{v}_1\,\mathbf{k}_1^\top \quad \Longrightarrow \quad \mathbf{y}_1 \neq \mathbf{y}_3 \quad \text{（兩者都應該是「old」）}$$

DeltaNet 的改進：寫入前先讀取 state 中 $\mathbf{k}_1$ 方向的舊值，只寫入差量（error correction）：

$$\mathbf{S}_3^{\text{DeltaNet}} = \mathbf{S}_2 + \mathbf{v}_1\,\mathbf{k}_1^\top - \underbrace{\hat{\mathbf{v}}_1}_{\mathbf{S}_2\,\mathbf{k}_1\text{：舊值}}\,\mathbf{k}_1^\top = \mathbf{S}_2 + \underbrace{(\mathbf{v}_1 - \hat{\mathbf{v}}_1)}_{\text{error}}\,\mathbf{k}_1^\top$$

	Linear Transformer	**DeltaNet**
行為	無論如何對 $\mathbf{S}$ 加 $\mathbf{v}_t\,\mathbf{k}_t^\top$	計算如何更新 $\mathbf{S}$ 使得 $\mathbf{S}\,\mathbf{k}_t = \mathbf{v}_t$
寫入前	不讀取，直接加 $\mathbf{v}_t\,\mathbf{k}_t^\top$	先讀取 $\hat{\mathbf{v}}_t = \mathbf{S}_{t-1}\,\mathbf{k}_t$（已在 $\mathbf{S}$ 中）
寫入內容	完整的 $\mathbf{v}_t\,\mathbf{k}_t^\top$（無條件疊加）	只寫修正量 $\mathbf{v}_t\,\mathbf{k}_t^\top - \hat{\mathbf{v}}_t\,\mathbf{k}_t^\top$

補充：當 $\mathbf{S} = \mathbf{v}_t\,\mathbf{k}_t^\top$ 且 $\|\mathbf{k}_t\| = 1$ 時，$\mathbf{S}\,\mathbf{k}_t = \mathbf{v}_t\,\mathbf{k}_t^\top\,\mathbf{k}_t = \mathbf{v}_t$。這就是「state matrix = key-value memory」的具體含義：用 key 查詢 state 可以取回對應的 value。

Deriving DeltaNet's State Update — Error Correction Form

From the "same old same old" concrete example, derive DeltaNet's general state update equation.

Step 1: Recall the DeltaNet update at $t = 3$ (concrete example):

$$\mathbf{S}_3 = \mathbf{S}_2 + \mathbf{v}_1 \mathbf{k}_1^\top - \hat{\mathbf{v}}_1 \mathbf{k}_1^\top, \quad \hat{\mathbf{v}}_1 = \mathbf{S}_2 \mathbf{k}_1$$

Step 2: Merge $\mathbf{v}_1$ and $\hat{\mathbf{v}}_1$ terms:

$$\mathbf{S}_3 = \mathbf{S}_2 + (\mathbf{v}_1 - \hat{\mathbf{v}}_1) \mathbf{k}_1^\top = \mathbf{S}_2 + (\mathbf{v}_1 - \mathbf{S}_2 \mathbf{k}_1) \mathbf{k}_1^\top$$

Step 3: Replace concrete subscripts with general form ($\mathbf{v}_1 \to \mathbf{v}_t$, $\mathbf{k}_1 \to \mathbf{k}_t$, $\mathbf{S}_2 \to \mathbf{S}_{t-1}$):

$$\mathbf{S}_t = \mathbf{S}_{t-1} + (\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t) \mathbf{k}_t^\top$$

$$\boxed{\mathbf{S}_t = \mathbf{S}_{t-1} + \underbrace{\beta_t}_{\text{learning rate}} (\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t) \mathbf{k}_t^\top}$$

Dynamic Learning Rate β_t

$\beta_t = \sigma(\mathbf{W}_\beta \mathbf{x}_t) \in (0, 1)$, where σ is the sigmoid function and $\mathbf{W}_\beta$ is a learned projection. When $\beta_t = 1$, the old value at $\mathbf{k}_t$ is fully replaced; when $\beta_t \to 0$, the state is preserved unchanged.

Deriving DeltaNet's State Update — Householder Matrix Form

From Step 3's error correction form, rewrite as Householder matrix form.

Step 3 (recap):

$$\mathbf{S}_t = \mathbf{S}_{t-1} + (\mathbf{v}_t - \mathbf{S}_{t-1}\,\mathbf{k}_t)\,\mathbf{k}_t^\top$$

Step 4: Expand brackets, factor out $\mathbf{S}_{t-1}$, rewrite as Householder form:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\,\mathbf{k}_t^\top - \mathbf{S}_{t-1}\,\mathbf{k}_t\,\mathbf{k}_t^\top$$

$$= \mathbf{S}_{t-1}\ \underbrace{(\mathbf{I} - \mathbf{k}_t\,\mathbf{k}_t^\top)}_{\text{Householder matrix}} + \underbrace{\mathbf{v}_t\,\mathbf{k}_t^\top}_{\text{write new}}$$

Adding learning rate $\beta_t \in (0, 1]$:

$$\boxed{\mathbf{S}_t = \mathbf{S}_{t-1}\,(\mathbf{I} - \underbrace{\beta_t}_{\text{learning rate}}\,\mathbf{k}_t\,\mathbf{k}_t^\top) + \beta_t\,\mathbf{v}_t\,\mathbf{k}_t^\top}$$

- $(\mathbf{I} - \beta_t\,\mathbf{k}_t\,\mathbf{k}_t^\top)$: **Householder matrix** — precisely erases old memory in $\mathbf{k}_t$ direction, other directions untouched
- $\beta_t\,\mathbf{v}_t\,\mathbf{k}_t^\top$: writes new key-value association

Householder matrix: https://en.wikipedia.org/wiki/Alston_Scott_Householder

State Update Equation — Linear Transformer vs DeltaNet

Linear Transformer (ICML 2020)

State Update:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$$

Decoding:

$$\mathbf{y}_t = \mathbf{S}_t \mathbf{q}_t$$

Write:

$$\mathbf{v}_t \mathbf{k}_t^\top$$

(full value, unconditional)

DeltaNet (ICML 2021)

State Update:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \beta_t \left(\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t\right) \mathbf{k}_t^\top$$

Decoding:

$$\mathbf{y}_t = \mathbf{S}_t \mathbf{q}_t$$

Write:

$$\beta_t \left(\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t\right) \mathbf{k}_t^\top$$

(error correction)

Summary:

- Decoding equation is the same: $\mathbf{y}_t = \mathbf{S}_t \mathbf{q}_t$
- DeltaNet only modifies the **write** method — from "unconditionally add $\mathbf{v}_t \mathbf{k}_t^\top$" to "read first, then correct the difference"

Supplementary: State Matrix = Key-Value Memory

Key insight: The state matrix S is a **key-value memory** that maps keys to values.

Two fundamental operations on S:

Operation	Formula	Meaning
Write (key k, value v)	$S \mathrel{+}= v k^\top$	Store association "$k \mapsto v$"
Read (key k)	$\bar{v} = S \cdot k$	Retrieve the value at "address" k

Decoding = Read: $\mathbf{y}_t = \mathbf{S}_t \cdot \mathbf{q}_t = \left(\sum_{j=1}^{t} \mathbf{v}_j \mathbf{k}_j^\top\right) \mathbf{q}_t = \sum_{j=1}^{t} (\mathbf{k}_j^\top \mathbf{q}_t)\, \mathbf{v}_j$

The problem reframed:

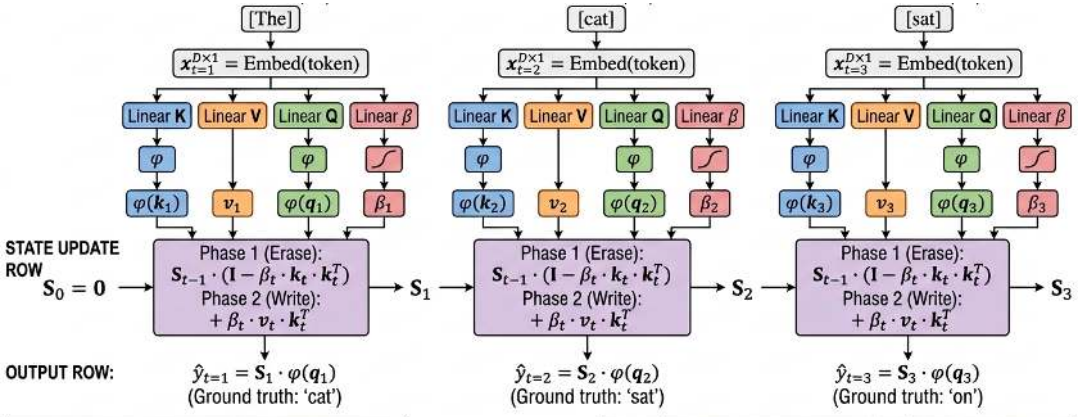
Linear Attention's S = Append-Only Log
(INSERT only, no UPDATE, no DELETE)
We need S = Key-Value Database (READ $\rightarrow$ UPDATE $\rightarrow$ OVERWRITE)

Correctness constraint — the goal of every write should be:

$$S_t \cdot k_t = v_t \quad \text{(ensure address } k_t \text{ stores value } v_t\text{)}$$

DeltaNet — State Matrix Recurrence

Example: "The cat sat on the mat ..."



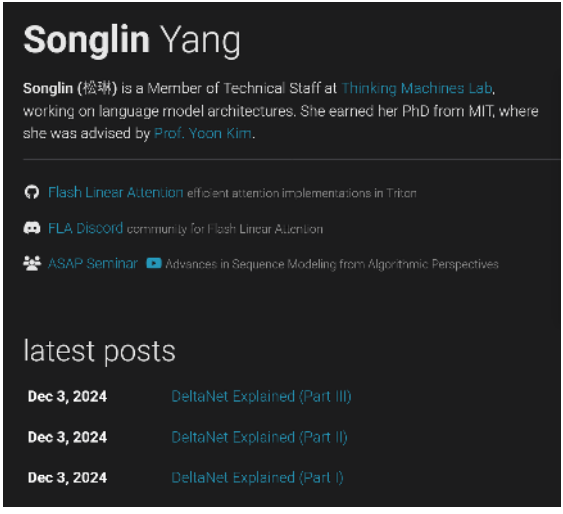
DeltaNet Recurrence (Delta Rule):

$$S_t = S_{t-1} \cdot (I - \beta_t \cdot k_t \cdot k_t^T) + \beta_t \cdot v_t \cdot k_t^T \quad \text{(gap)}$$

$$\hat{y}_t = S_t \cdot \varphi(q_t)$$

$\varphi(q_t)$	Transformed query	v_t	Value vector
$\varphi(k_t)$	Transformed key (L2 norm)	S_t	State matrix, $d_k \times d_v$
β_t	Learning rate (per-step)	$\hat{y}_t$	Output (prediction)

Songlin Yang's DeltaNet Blog (Gated DeltaNet Author)



https://sustcsonglin.github.io/

4. Gated DeltaNet: Combining DeltaNet + Mamba-2

1. Qwen3.5 Overview
2. Linear Transformer
3. DeltaNet: Delta Rule + Householder Erasure
▶ **4. Gated DeltaNet: Combining DeltaNet + Mamba-2**
5. Comparison of Three Methods
6. Limitations of Linear Attention Layers
7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

Gated DeltaNet — The Best-of-Both-Worlds Approach

"Gated Delta Networks: Improving Mamba2 with Delta Rule" (ICLR 2025)

DeltaNet:

$$S_t = S_{t-1} \cdot (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

Gated DeltaNet (adds α_t global decay from Mamba-2):

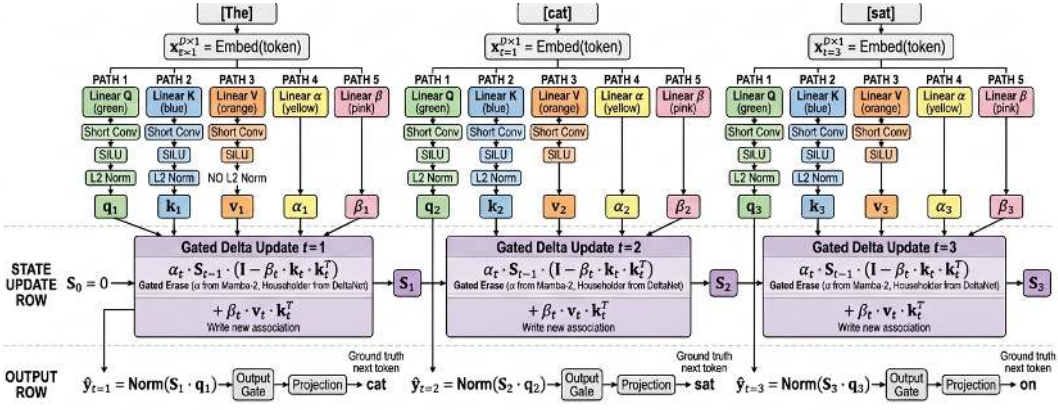
$$S_t = \underbrace{\alpha_t \cdot S_{t-1}}_{\text{Global Decay (Mamba-2)}} \cdot \underbrace{(I - \beta_t k_t k_t^\top)}_{\text{Targeted Erasure (DeltaNet)}} + \underbrace{\beta_t v_t k_t^\top}_{\text{Write New (DeltaNet)}}$$

Three memory operations unified in a single equation:

Operation	Formula Component	Origin	Description
Global Decay	$\alpha_t \cdot S_{t-1}$	Mamba-2	$\alpha_t \in (0, 1)$, controls "how much old memory to forget"
Targeted Erasure	$(I - \beta_t k_t k_t^\top)$	DeltaNet	Householder matrix, erases old memory in k_t direction
Write New	$\beta_t v_t k_t^\top$	DeltaNet	Writes new key-value association into state matrix

Gated DeltaNet — Recurrence Update with Gated Delta Rule

Example: "$\underbrace{\text{The}}_{t=1}$ $\underbrace{\text{cat}}_{t=2}$ $\underbrace{\text{sat}}_{t=3}$ on the mat ..."



Gated Delta Rule (ICLR 2025):

$$\mathbf{S}_t = \alpha_t \cdot \mathbf{S}_{t-1} \cdot (\mathbf{I} - \beta_t \cdot \mathbf{k}_t \cdot \mathbf{k}_t^T) + \beta_t \cdot \mathbf{v}_t \cdot \mathbf{k}_t^T$$

$$\hat{\mathbf{y}}_t = \text{OutputGate}(\text{Norm}(\mathbf{S}_t \cdot \mathbf{q}_t))$$

$\mathbf{q}_t$	Query (L2 normalized)	$\mathbf{v}_t$	Value vector
$\mathbf{k}_t$	Key (L2 normalized)	$\mathbf{S}_t$	State matrix, $d_k \times d_v$
α_t	Gating scalar (Mamba-2)	β_t	Learning rate (DeltaNet)

Mamba-2 Elements in Gated DeltaNet

The Gated DeltaNet paper is titled *"Improving **Mamba2** with Delta Rule"*—it is built directly on the Mamba-2 framework.

Element	From Mamba-2	Role in Gated DeltaNet
Exponential Gating α_t	✓	Global decay of the state matrix, controls "how much old memory to forget"
Short Conv (Causal Conv1D)	✓	Local context modeling after Q/K/V projections
SSD Duality (SSM = Linear Attention)	✓	Matrix multiplication during training, recurrence during inference
Chunkwise Training Algorithm	✓	Extends Mamba-2's chunked training method

Mamba-2: $S_t = \alpha_t \cdot S_{t-1} + v_t k_t^\top$

Gated DeltaNet adds the Delta Rule on top of this:

$$S_t = \alpha_t \cdot S_{t-1} \cdot (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

For detailed Mamba-2 technical content, please refer to the supplementary slides at the end of this presentation.

Summary of DeltaNet and Gated DeltaNet

1. **DeltaNet** (Schlag et al., ICML 2021) — 先讀再寫，只補差量

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \beta_t \left(\mathbf{v}_t - \mathbf{S}_{t-1}\,\mathbf{k}_t\right)\mathbf{k}_t^\top$$

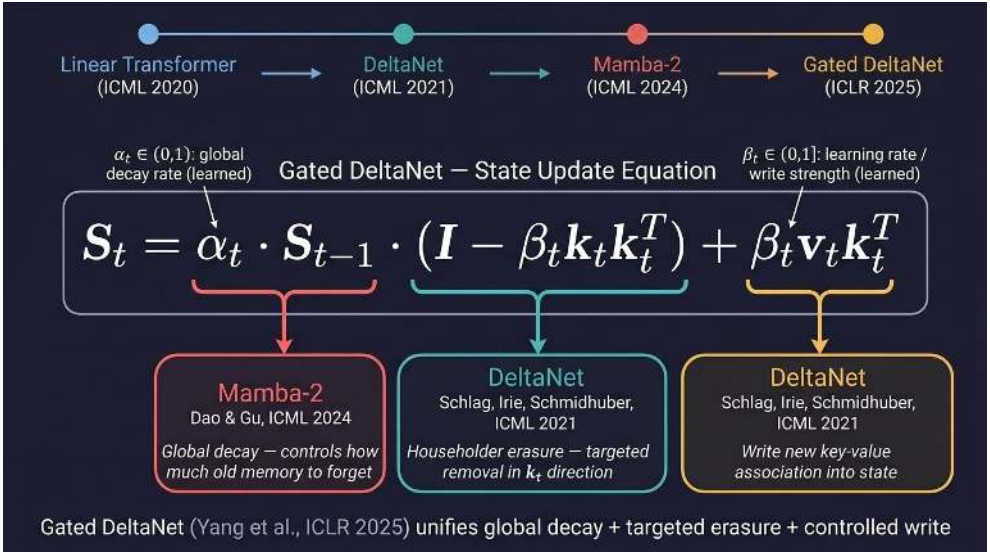
2. **Gated DeltaNet** (Yang et al., ICLR 2025) — 全域衰減，淡化舊記憶

$$\mathbf{S}_t = \alpha_t \cdot \mathbf{S}_{t-1} \cdot \left(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top\right) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

5. Comparison of Three Methods

1. Qwen3.5 Overview
2. Linear Transformer
3. DeltaNet: Delta Rule + Householder Erasure
4. Gated DeltaNet: Combining DeltaNet + Mamba-2
▶ **5. Comparison of Three Methods**
6. Limitations of Linear Attention Layers
7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

Gated DeltaNet — Equation Ancestry



State Update Equation Evolution

Linear Transformer → DeltaNet → Mamba-2 → Gated DeltaNet

Equation 1 — Linear Transformer (Katharopoulos et al., ICML 2020):

$$S_t = S_{t-1} + v_t k_t^\top$$

Blind accumulation — can only ADD, never erase or forget. Cannot modify old associations.

Equation 2 — DeltaNet (Schlag et al., ICML 2021):

$$S_t = S_{t-1} \cdot (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

NEW: Erase old association in k_t direction via Householder Erasure, then write. β_t controls both. **But:** no global forgetting.

Equation 3 — Mamba-2 (Dao & Gu, ICML 2024):

$$S_t = \alpha_t \cdot S_{t-1} + v_t k_t^\top$$

NEW: α_t globally decays all memory. Proved SSM = Linear Attention (SSD Duality). **But:** cannot precisely update specific key-value.

Equation 4 — Gated DeltaNet (Yang et al., ICLR 2025):

$$\boxed{S_t = \alpha_t \cdot S_{t-1} \cdot (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top}$$

Combines: α_t global decay + targeted erasure + controlled write. Can both forget AND precisely update!

Comparison of Three Methods — Equations & Components

Side-by-side equation comparison

Method	State Update Equation
Linear Transformer (2020)	$S_t = S_{t-1} + v_t k_t^\top$
DeltaNet (2021)	$S_t = S_{t-1} \cdot (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$
Mamba-2 (2024)	$S_t = \alpha_t \cdot S_{t-1} + v_t k_t^\top$
Gated DeltaNet (2025)	$S_t = \alpha_t \cdot S_{t-1} \cdot (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$

Component	Linear Transformer	DeltaNet	Mamba-2	Gated DeltaNet
State Matrix S_t	✓	✓	✓	✓
Global Decay α_t	✗	✗	✓	✓
Householder Erasure $(I - \beta_t k_t k_t^\top)$	✗	✓	✗	✓
Learning Rate β_t Write Control	✗	✓	✗	✓
Short Conv (Causal Conv1D)	✗	✗	✓	✓
L2 Norm on Q/K	✗	✗	✗	✓
SiLU Activation	✗	✗	✗	✓
Output Gate + Norm	✗	✗	✗	✓

6. Limitations of Linear Attention Layers

1. Qwen3.5 Overview
2. Linear Transformer
3. DeltaNet: Delta Rule + Householder Erasure
4. Gated DeltaNet: Combining DeltaNet + Mamba-2
5. Comparison of Three Methods
6. **Limitations of Linear Attention Layers**
7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

Why Not 100% Gated DeltaNet?

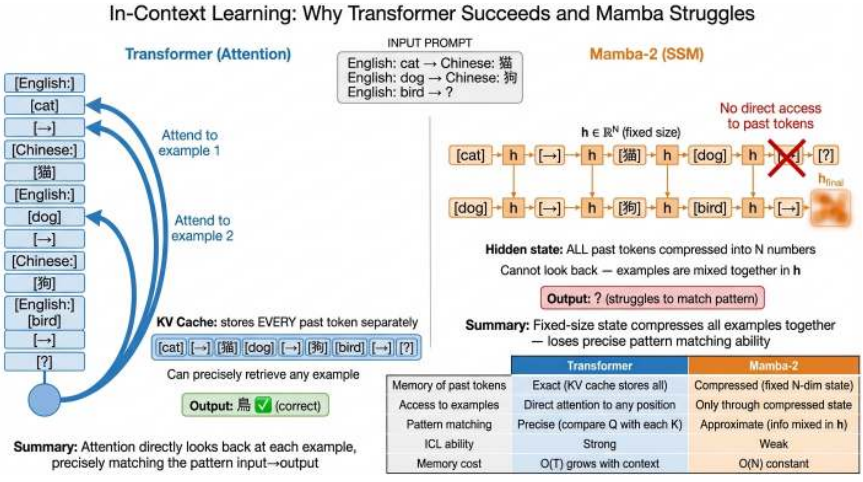
- Pure Gated DeltaNet still underperforms full attention on **precise in-context retrieval**
- Qwen3.5's solution: **3:1 hybrid** — 75% Gated DeltaNet + 25% Full Softmax Attention
  - Inference throughput: **8.6x**↑ at 32K context, **19x**↑ at 256K context (vs pure Softmax)

Model	SWDE	SQD	FDA	TQA	NQ	Drop	Avg
DeltaNet	17.9	30.9	18.4	53.9	17.3	18.6	26.2
Mamba2	19.1	33.6	25.3	61.0	20.8	19.2	29.8
Gated DeltaNet	25.4	34.8	23.7	60.0	19.0	19.8	30.6
Transformer++	29.5	38.0	52.2	58.3	22.5	21.6	37.0
Gated DeltaNet-H1 (1:1)	35.6	39.7	52.0	60.1	24.6	22.5	39.0
Gated DeltaNet-H2 (3:1)	**38.2**	**40.4**	**50.7**	**63.3**	**24.8**	**23.3**	**40.1**

H1 = 1:1 (50% Gated DeltaNet + 50% Attn) **H2** = 3:1 (75% Gated DeltaNet + 25% Attn)
Pure Gated DeltaNet (30.6) < Transformer++ (37.0), but **Gated DeltaNet-H2 (40.1) surpasses it (+3.1)**

Source: Yang et al., *Gated Delta Networks*, ICLR 2025, Table 4 (1.3B params, 2K tokens)

In-Context Learning: Transformer vs Linear Attention

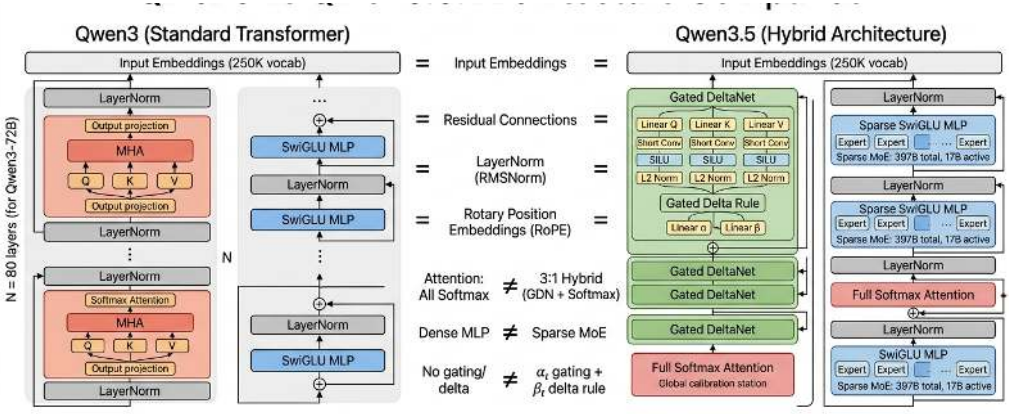


Pure linear attention models struggle with precise in-context retrieval; interleaving softmax attention layers recovers this capability while preserving the efficiency gains of linear layers.

7. Qwen3.5 and Qwen3-Next: Hybrid Architecture

1. Qwen3.5 Overview
2. Linear Transformer
3. DeltaNet: Delta Rule + Householder Erasure
4. Gated DeltaNet: Combining DeltaNet + Mamba-2
5. Comparison of Three Methods
6. Limitations of Linear Attention Layers
▶ **7. Qwen3.5 and Qwen3-Next: Hybrid Architecture**

Gated DeltaNet Deployment in Qwen3.5 — Compared with Qwen3



Component	Qwen3	Qwen3.5
Attention	100% Softmax	75% Gated DeltaNet + 25% Softmax
MLP	Dense	Sparse MoE (397B/17B)
Complexity	$O(N^2)$ all layers	$O(N)$ for 75% layers, $O(N^2)$ for 25%
Context	32K	262K (expandable to 1M)
Params	72B (all active)	397B total, 17B active

- **Architecture Foundation**: Qwen3-Next (first adopted Gated DeltaNet in Sep 2025)

Supplement — Qwen3.5 vs Qwen3.6-Plus — Specifications

	Qwen3.5 (397B-A17B)	**Qwen3.6-Plus**
Release Date	Feb 16, 2026	Mar 30, 2026 (preview)
License	Apache 2.0 (open weights)	Closed source (API only)
Parameters	397B total, 17B active	$\sim$480B total, $\sim$24B active*
Architecture	Gated DeltaNet + MoE (3:1 hybrid)	Linear attention + MoE hybrid
Context	262K (extensible to 1M)	Native 1M
Modalities	Text, image, video, audio	Text, image
Positioning	General-purpose multimodal	Agentic coding
Thinking Mode	Optional	Always on
Reasoning Efficiency	91% tokens on reasoning	83% tokens on reasoning
Tool Integration	—	Claude Code, Cline

*Qwen3.6-Plus parameter counts are community estimates, not officially confirmed.

Recent Large Language Model Comparison (2026-05-15 Updated)

	Claude Opus 4.6	**Qwen3.6 -Plus**	**Gemma 4 31B**	**Kimi K2.6**	**DeepSeek V4-Pro**
Organization	Anthropic	Alibaba	Google	Moonshot AI	DeepSeek
Release	Feb 2026	Mar 2026	Apr 2, 2026	Apr 20, 2026	Apr 24, 2026
Type	Proprietary	MoE+GDN	31B Dense	1T MoE	1.6T MoE
Reasoning					
GPQA Diamond	**91.3**	88.4†	84.3	90.5	90.1
AIME 2026	93.3	91.3†	89.2	96.4	**99.4**
Coding					
SWE-bench Verified	**80.8**	78.8	52.0	80.2	80.6
LiveCodeBench	—	83.6†	80.0	89.6	**93.5**

†Qwen3.5-397B-A17B score used where Qwen3.6-Plus score is not publicly available.

- Open-weight models now within **1 point** of proprietary leaders on SWE-bench Verified
- **DeepSeek V4-Pro** (1.6T/49B active) dominates coding: LiveCodeBench 93.5, AIME 99.4
- MMLU is saturated (88–99% for all); GPQA Diamond is the new discriminator

Summary and Takeaways

Summary

- Linear Transformer: $O(L^2) \rightarrow O(Ld^2)$ via kernel ϕ
- DeltaNet: delta rule fixes blind accumulation
- Gated DeltaNet = α_t decay + β_t delta rule
- 3:1 hybrid surpasses Transformer++
- Qwen3.5 deploys Gated DeltaNet at scale

Next Week

- Qwen3.5 native multimodal (Early Fusion)
- Gemma 4 Per-Layer Embeddings (PLE)

Takeaways

1. Step-by-step derivation from standard attention to state matrix — making the math accessible
2. Concrete example showing **why** blind accumulation fails
3. Unified view: Linear Transformer → DeltaNet → Gated DeltaNet as incremental fixes
4. Practical insight: pure linear attention is not enough — hybrid 3:1 is the sweet spot

Thank You for Listening!

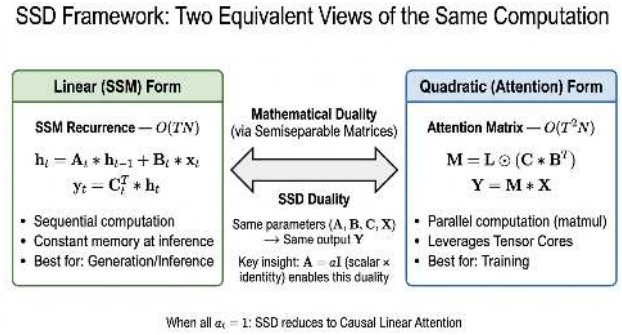
Questions & Discussion

Supplementary Slides

Supplementary Slide A: Mamba-2 Contribution 1 — SSD Duality

Contribution 1: SSD Duality — Proving SSM = Linear Attention

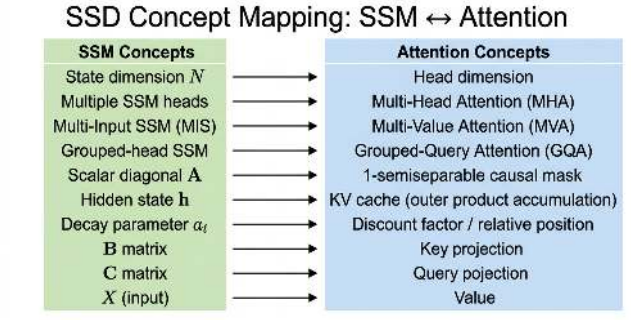
- Mamba-2 (Dao & Gu, ICML 2024) proved that State Space Models and Linear Attention are mathematically equivalent
- This means: **use matrix multiplication during training** (fast, fully utilizing GPUs), **use recurrence during inference** (memory-efficient)
- This equivalence provided the theoretical foundation for Gated DeltaNet's cross-tradition fusion



Supplementary Slide A: Mamba-2 Contribution 2 — Exponential Gating

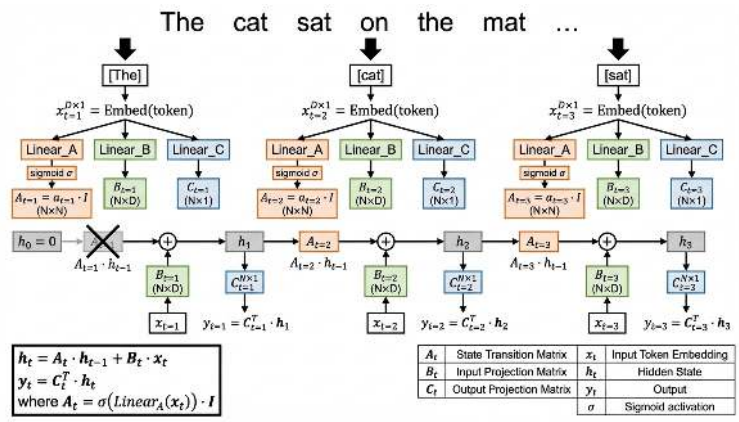
Contribution 2: Exponential Gating Mechanism (α_t)

- Introduces gated decay in the state matrix update: $S_t = \alpha_t \cdot S_{t-1} + \phi(k_t)v_t^\top$
- $\alpha_t \in (0, 1)$: Controls "how much old memory to forget"
- $\alpha_t \to 0$: Nearly clears all old memory (context switching)
- $\alpha_t \to 1$: Nearly retains all old memory



Note: When $\alpha_t = 1$ for all t: SSM reduces to Causal Linear Attention with standard causal mask

Supplementary Slide B: Mamba-2 SSM Operation — Diagram



- Recurrent form of SSM: $h_t = A_t \cdot h_{t-1} + B_t \cdot x_t, \quad y_t = C_t^\top \cdot h_t$
- Where $A_t = \sigma(\text{Linear}_A(x_t)) \cdot I$ is a data-dependent diagonal decay matrix

Supplementary Slide B: Mamba-2 SSM Operation — Implications

- SSD Duality proved that this SSM recurrence and the Linear Attention state matrix update are mathematically equivalent
- **But the problem with Mamba-2**: α_t can only **uniformly decay all memory**, and cannot precisely update specific key-value associations

Supplementary: DeltaNet — Conversion Condition from Quadratic to Linear

As long as $\text{sim}(q, k) = \phi(q)^\top \phi(k)$, we can use associativity to convert $O(L^2)$ to $O(Ld^2)$. But how exactly should we choose ϕ? DeltaNet provides a concrete technical path.

Necessary conditions for the conversion:

Condition	Explanation	Why Softmax fails
Finite-dim feature map	$\phi(x) \in \mathbb{R}^{d'}$, d' must be finite and controllable	Taylor expansion of $\exp(q^\top k)$
Non-negativity (or handling negatives)	Accumulated matrix $S = \sum \phi(k) v^\top$ needs numerical stability	—
Sufficient expressiveness	ϕ must be able to distinguish different keys	—

Supplementary: DeltaNet — Specific Conversion Method

DeltaNet's specific conversion method:

1. **Kernel function choice**: Use a ReLU variant* as ϕ
   - Concatenate $[k; -k] \rightarrow$ ReLU $\rightarrow$ cross-product
   - Dimensions expand from d to $2d\nu$, increasing memory capacity
   - Satisfies finite-dimensionality + non-negativity + information preservation
2. **Delta Rule compensates for expressiveness loss**: Even with a weaker kernel, a precise memory update strategy can maintain performance
3. **Subsequent evolution**: Delta rule naturally supports **negative values**, so SiLU replaced the ReLU variant*, removing the non-negativity constraint

Key insight: The kernel function (ϕ) and the memory update strategy (delta rule) must be **co-designed** to maintain model quality while reducing complexity.

* ReLU variant refers to DPFP (Deterministic Parameter-Free Projection); see Schlag et al., ICML 2021.

Supplementary: DeltaNet's Three Major Contributions to Qwen3.5

1. **Delta Rule memory update mechanism**: $S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top$ — replacing blind accumulation.
2. **Kernel function design methodology**: DPFP $\rightarrow$ ELU+1 $\rightarrow$ SiLU evolution.
3. **Householder matrix insight**: $S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ — precise erasure in the k_t direction. Mathematical foundation for Gated DeltaNet.